Mark	

Team name	*A10*
Homework number	*HOMEWORK 09*
Due date	20/11/2026

Contribution	NO	Partial	Full
Carbone Alessandro			*X*
Casareale Alberto			*X*
Colombi Federico			*X*
Lorusso Luca			*X*
Martorano Gabriele			*X*
Notes: Everyone contributed to understanding the logic and implementing the code			

Project name	Accelerometer over I2C (interrupt and DMA)		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Configuration and Routing

To achieve the objective of reading the acceleration from the LIS2DE sensor over I2C and sending it to a remote terminal every 1 second, we began by verifying the hardware connections on the board, noting that the accelerometer uses the I2C1 bus. In STM32CubeMX, we configured the I2C1 pins PB8 (I2C1_SCL) and PB9 (I2C1_SDA) in Standard Mode and USART2 in Asynchronous mode. To ensure maximum efficiency and prevent CPU stalling, we employed DMA. In the DMA Settings, we added requests for both I2C1_RX (to receive sensor data) and USART2_TX (to send the formatted string). Crucially, in the NVIC Settings, we enabled the I2C1 Event and Error interrupts, the USART2 global interrupt, and the DMA stream global interrupts. To dictate the strict 1-second timing, Timer 2 (TIM2) was configured with its global interrupt enabled.

Part 2: Code Generation and Verification

After generating the project, we verified the timing parameters for TIM2 in the main.c file. Given the 84 MHz system clock, setting the prescaler (PSC) to 8400-1 and the counter period (ARR) to 10000-1 ensures a precise 1-second interrupt interval. Because floating-point numbers are used to format the acceleration values, we explicitly added the -u _printf_float linker flag via target_link_options in the CMakeLists.txt file. In the main() function, prior to the infinite loop, we initialize the LIS2DE sensor to exit its default power-down state. We send the value 0x17 to the CTRL_REG1 (address 0x20 at the I2C device address 0x50),

effectively enabling the X, Y, and Z axes and setting the Output Data Rate (ODR) to 1 Hz. Finally, the system is set in motion by calling HAL_TIM_Base_Start_IT(&htim2), leaving the while(1) loop completely empty.

Part 3: Functional Logic Implementation

The application logic is entirely event-driven and runs in the background. Every second, TIM2 triggers the HAL_TIM_PeriodElapsedCallback. Inside this callback, an asynchronous DMA read is initiated using HAL_I2C_Mem_Read_DMA starting at register 0x28. By applying the bitwise OR 0x28 | 0x80, we activate the LIS2DE's auto-increment feature, allowing the DMA to fetch 6 consecutive bytes (3 axes plus 3 interleaved reserved bytes) in a single operation.

Once the I2C reception is complete, the HAL_I2C_MemRxCpltCallback is fired. Within this routine, the raw 8-bit, left-justified, two's complement data is extracted from the odd indices of the buffer and cast to int8_t to preserve the sign. The raw values are then multiplied by the sensor's sensitivity (0.0156 g/digit for a ±2g full scale) to obtain the acceleration in g. Finally, these float values are formatted into a string using snprintf and transmitted to the terminal via HAL_UART_Transmit_DMA, completing the cycle with zero CPU blocking.

Professor comments: